LEGION

WEBAPP CAPTURE · BEGINNER GUIDES

Make Demo Videos & Audit My UX/UI with Claude

Six step-by-step guides: install, your first demo video, screenshots and audits, finishing in ChatGPT, selling the work, and fixing problems.

Plugin version 2.0.0
Generated 2026-10-23
legioncodeinc.com

I am Legion. We are Legion.

Contents

1. Install everything

- What you need
- Step 1. Install the Claude desktop app
- Step 2. Open the Code tab on your project
- Step 3. Install this plugin
- Step 4. Ask Claude to install the free tools
- Step 5. Check that everything works
- Prefer a terminal?

2. Your first demo video

- Step 1. Start your app
- Step 2. Open the Code tab on your project
- Step 3. Check your computer
- Step 4. Ask for a demo
- Step 5. Answer Claude's questions
- Step 6. Log in once
- Step 7. Approve the plan
- Step 8. Claude records
- Step 9. Check the privacy review
- Step 10. Fix the timing (if needed)
- Step 11. Make the video file
- Need a retake?

3. Screenshots, library and audit

- Screenshots: a picture of every page
- Library: every component, documented
- Audit: what does not match
- shadcn: move your look to shadcn/ui
- Claude Design handoff
- Everything at once

4. Finish your video in ChatGPT

- What you need
- Step 1. In Claude: make the handoff zip
- Step 2. Install this plugin in ChatGPT (once)
- Step 3. Connect ElevenLabs (once)
- Step 4. Replicate token (only for AI-made intros)
- Step 5. In ChatGPT: finish the video
- Step 6. Check the final video
- Free B-roll sites

5. Make money with it

- How a job goes
- What to charge
- Fees you pay
- How to sell it

6. Fix problems

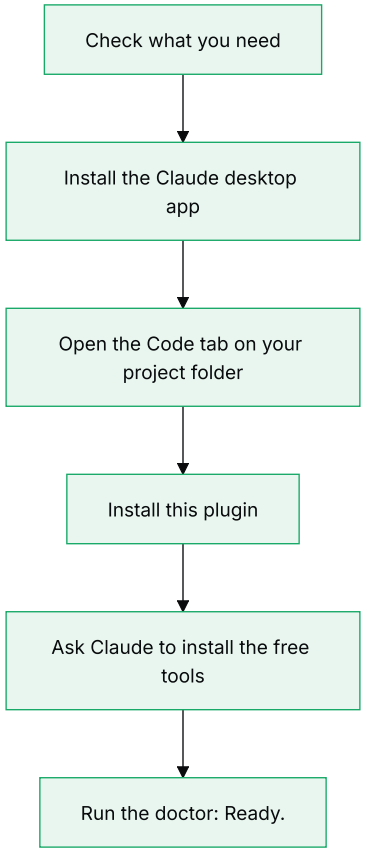
- Installing
- Logging in
- Demo videos
- ChatGPT, voice and intro
- Files and folders
- Still stuck?

7. Reference

- What it installs
- Demo route in depth
- How it stays safe
- Where outputs go

1. Install everything

You do this once. It takes 15 to 30 minutes. You will use the **Claude desktop app** the whole time. You do not need to type commands into a terminal: you ask Claude, and Claude does the technical parts.



What you need

Thing	Why	Cost
A Mac or Windows computer	Runs the Claude desktop app (Linux is in beta)	
A Claude **Pro, Max, Team or Enterprise** plan	The Code tab (Claude Code) is not included in the free plan (setup docs)	Paid Claude plan

Thing	Why	Cost
The app you want to record, running somewhere you can open in a browser	It is what gets captured	
For voice later: an ElevenLabs account	Makes the narrator voice	Free to try. **Selling the audio to clients needs a paid plan**: the free plan has no commercial license (ElevenLabs help)
For AI intro clips later (optional): a Replicate account	Makes intro videos	Pay per clip. See 4. Finish your video in ChatGPT

Step 1. Install the Claude desktop app

1. Go to the Claude desktop download page and download the app for your computer (Mac, or Windows x64 or ARM64).
2. Open the installer and follow it like any other app.
3. Open Claude and sign in with your Claude account.

Check: you see the Claude app with a **Code** tab at the top.

Step 2. Open the Code tab on your project

The **Code** tab is Claude Code inside the desktop app. It can read files and run tools on your computer, but only after you say yes.

1. Click the **Code** tab.
2. Before you type anything, click the **project folder** control (next to the environment selector) and choose the folder of the project you want to capture. If you do not have one, make an empty folder called `capture` in Documents and choose that.
3. Keep the environment on your own computer (local). Plugins do not run in cloud sessions.

You will see a chat box. Everything from here on is typed into that box.

About permission pop-ups: when Claude wants to install something or run a tool, the app asks you first. Read what it wants to do and click **Allow** if it matches what you asked for.

Step 3. Install this plugin

This is the one time you paste two lines into the app's built-in terminal panel. It takes a minute.

1. Click **Terminal** in the app's title bar (or press Ctrl and the backtick key, `` ` ``). A small panel opens inside the app.
2. Paste this line and press Enter. It tells Claude where Legion Code's plugins live:

```
claude plugin marketplace add legioncodeinc/app-auditor-plugin-claude
```

3. Paste this line and press Enter. It installs the plugin for your user, so it works in every project:

```
claude plugin install webapp-capture@legioncodeinc
```

4. Start a new session in the Code tab so the plugin loads.

Check: click the **+** button next to the chat box, then **Plugins**, then **Manage plugins**. You should see **webapp-capture** in the list. (Install plugins, desktop docs)

Why the terminal panel for this step: the desktop **Plugins** screen lists marketplaces you already added. Adding a new one from GitHub is documented with these two lines. Your plugin choice is shared by the desktop app and Claude Code everywhere, so you only do this once.

What gets installed: the `/webapp-capture` command, two skills (instructions Claude follows), a helper agent, and a small video tool connection (an MCP server) for Replicate. On first use, the plugin also downloads its script helpers (Playwright and sharp) and a copy of the Chromium browser. The full list is in What it installs.

Step 4. Ask Claude to install the free tools

The plugin needs three free tools: **Node.js** (runs the capture scripts), **ffmpeg** (builds the video files), and **Python** (only for the voice, intro and final stitch). Type:

```
Check whether Node.js 20.9 or newer, ffmpeg, and Python 3.11 or newer are installed.
Install any that are missing, explain each step in plain words, and ask me before
installing anything.
```

You will see Claude check each tool, then ask permission to install what is missing. On a Mac it usually uses Homebrew; on Windows it uses winget.

If your computer asks for your password: Claude cannot type passwords, and you should never paste one into the chat. Claude will tell you the exact line to run. Open the **Terminal** panel (title bar, or Ctrl and backtick), paste the line, and type your password there yourself.

Windows note: also ask Claude to install **Git for Windows** (git-scm.com). The desktop app uses Git, and its Git Bash runs the video assembly script (`make-video.sh`). The Replicate intro tools work only on Mac and Linux, so on Windows use free stock B-roll for intros. (Plugins do not load in WSL sessions, so stay in a normal local session.)

Step 5. Check that everything works

Type:

```
/webapp-capture doctor demo
```

You will see a list of checks marked `ok` , `WARN` or `ERROR` . Claude explains and fixes anything marked `ERROR` . If it asks to install the Chromium browser, click **Allow**.

Done when the list ends with `Ready.`

If stuck: see 6. Fix problems.

Prefer a terminal?

Everything above also works in Claude Code in a terminal. Install it with the official setup docs, type `claude` in your project folder, and use the same prompts. Install the plugin there with `claude plugin marketplace add legioncodeinc/app-auditor-plugin-claude` and `claude plugin install webapp-capture@legioncodeinc` .

What's next

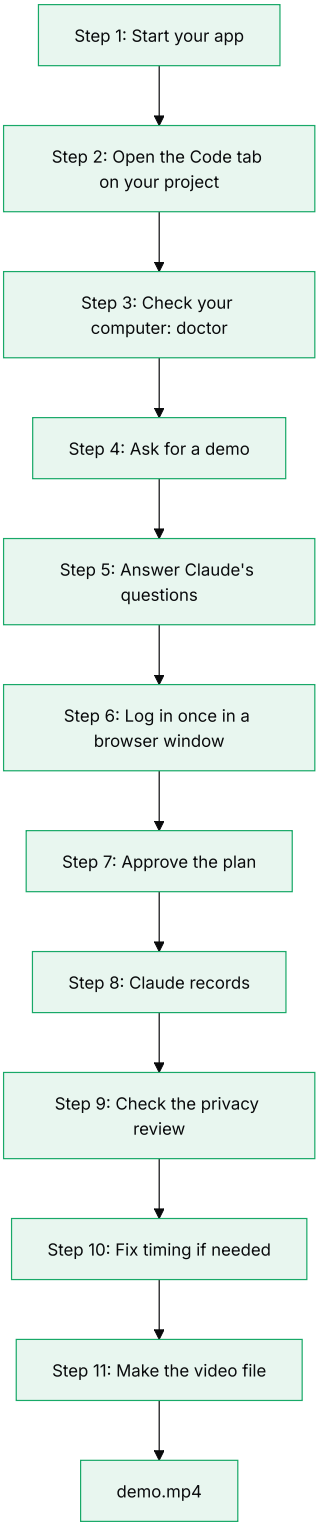
Make your first video: 2. Your first demo video.

2. Your first demo video

In this guide you make a 60 to 90 second video of your app, with captions, from start to finish. It takes about 30 to 60 minutes the first time. Most of that is Claude working while you wait.

Before you start: finish 1. Install everything. You need the Claude desktop app, this plugin, and ffmpeg.

The whole trip


Step 1: Start your app
Step 2: Open the Code tab on your project
Step 3: Check your computer: doctor
Step 4: Ask for a demo
Step 5: Answer Claude's questions
Step 6: Log in once in a browser window
Step 7: Approve the plan
Step 8: Claude records
Step 9: Check the privacy review
Step 10: Fix timing if needed
Step 11: Make the video file
demo.mp4

Step 1. Start your app

The plugin records your app the way a visitor sees it, so the app has to be running and reachable in a web browser.

- **If you build the app on your computer:** start it the way you normally do. Not sure how? Ask Claude in the Code tab: "start my app and tell me its address". It usually shows an address like `http://localhost:3000`. "localhost" means "this computer". Write the address down.
- **If the app is already online:** use a test copy if you have one, such as `https://staging.yourapp.com`. Recording your real live site works too, but then real customer data can show up on screen.
- **Use a test account with sample data.** Empty accounts make boring videos, and real customer names should not end up in a video.

Check: open the address in your normal web browser. You should see your app.

Step 2. Open the Code tab on your project

1. Open the Claude desktop app and click the **Code** tab.
2. Before you type anything, click the **project folder** control and choose your project's folder (see 1. Install everything, step 2).
3. Start a new session.

You will see a chat box. Everything from here on is typed into that box.

Step 3. Check your computer

Type:

```
/webapp-capture doctor demo
```

You will see a list of checks marked `ok`, `WARN` or `ERROR`, each with a fix. Claude fixes most of them for you. If it asks to install something (like Chromium, the browser it records with), click **Allow**.

Done when the list ends with `Ready.`

Step 4. Ask for a demo

Type `/webapp-capture demo`, your app's address, and what the video should show, in normal words:

```
/webapp-capture demo http://localhost:3000 a 60 second video for new customers showing how to book a job
```

More examples are in Common prompts.

Step 5. Answer Claude's questions

Claude asks everything it needs in one round. Typical questions:

Question	What to answer
Which environment is this?	local, staging, a demo account, or production
What page opens right after you log in?	The real one, for example `/schedule`
Light or dark theme?	What you want the video to show
Where should files go?	The default, `library/design/`, is fine
What should be hidden?	Anything private: customer names, emails, keys, billing
Who is the video for, and how long?	For example: new customers, 60 seconds
What must it show?	The 2 or 3 things that matter most
Does the demo change any data?	For example: books one appointment. Say no if it should only look around.

Step 6. Log in once

Claude opens a real browser window at your app's login page.

1. Log in yourself. Two-step codes and "Sign in with Google" work.
2. Do not close the window. It closes by itself once you are in.

Why: Claude never sees or types your password. It only saves the "you are logged in" cookie, in a private file that is kept out of git.

If stuck: if the window says it timed out, ask Claude to "run the login step again".

Step 7. Approve the plan

Claude runs a small test, then shows you:

- **the script**: what the narrator will say in each scene,
- **the plan**: each scene's page and actions (click, type, drag, scroll), and
- **the writes**: every change to your data, with what should happen. For example: "Creates one appointment on Tuesday, assigned to Sam".

Read them. Ask for changes in normal words ("make scene 2 shorter", "do not open the billing page"). When it looks right, type **yes, approved**. Nothing records before you approve, and each data change needs your yes.

Step 8. Claude records

Each scene is recorded in its own browser, one after the other. This takes a few minutes. You do not need to watch.

After every data change, Claude checks the page to prove it worked. If a check fails, Claude stops making more changes, takes a screenshot, and writes it down in `findings.md` for you to look at. Scenes that only look around keep going.

You will see a short report: which scenes passed, which failed, and where the files are.

Step 9. Check the privacy review

Type:

```
Run the privacy review on the demo
```

Claude makes a contact sheet (one picture with a grid of frames) for each scene, and a list of any emails or phone numbers that showed on screen, masked like `d**@**.com`.

1. Open the `review` folder inside your demo folder.
2. Look at every picture. Watch for names, emails, phone numbers and pop-ups.
3. If something private shows, tell Claude what to hide, then ask for a retake (see below).

Step 10. Fix the timing (if needed)

Claude checks if each narration line fits in its scene. If one is too long, it says **OVERRUN**. Type something like:

```
Narration is too long in scene 2. Shorten the line and rebuild the captions.
```

Step 11. Make the video file

Type:

```
Make the final video file
```

Claude runs `make-video.sh`, which joins the scenes, holds the last frame for 1.5 seconds, and adds captions.

You will find `demo.mp4` in `library/design/demo/<your-video-title>/`. Double-click it to watch. Captions can be turned on in your video player.

Need a retake?

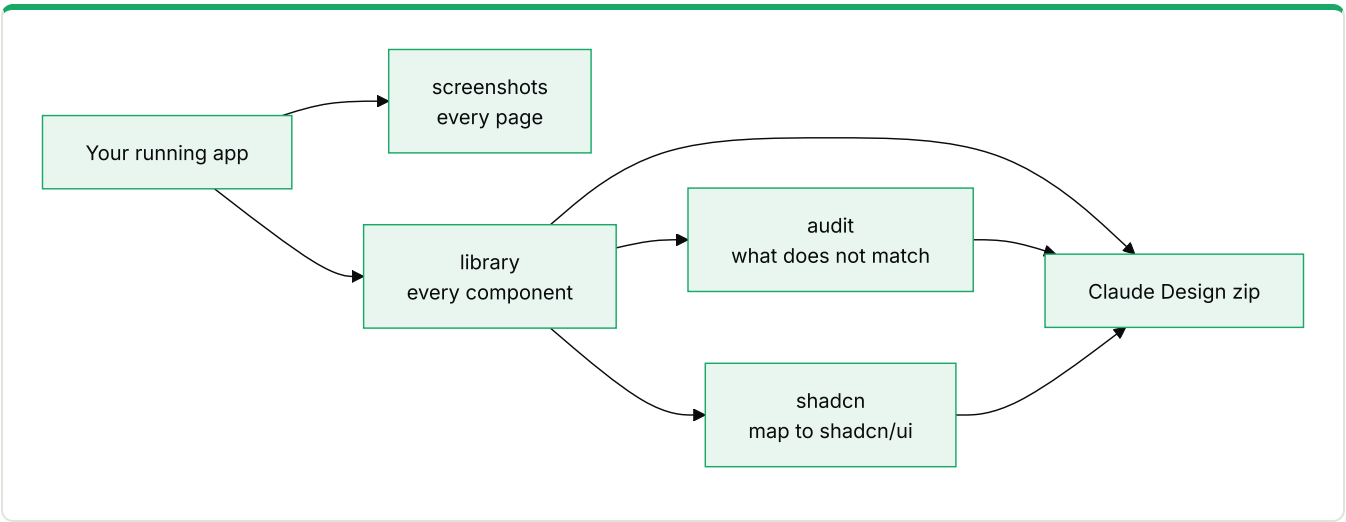
Type `Retake the demo. I reset the test data.` Each take goes in its own folder, so nothing is overwritten. If the first take already made data changes, Claude lists them and asks before making them again. Reset your test data first, or let Claude pick new free slots.

What's next

- Add a voice and an intro clip in ChatGPT: 4. Finish your video in ChatGPT
- Something went wrong: 6. Fix problems

3. Screenshots, library and audit

The plugin can do more than videos. This guide covers the other four jobs. They start the same way as a demo: your app is running, you open the **Code** tab of the Claude desktop app on your project folder, and you type a `/webapp-capture` command. Steps 1 to 6 of 2. Your first demo video (start the app, open the Code tab, doctor, ask, answer questions, log in) are the same here.



These jobs only look at your app. They open pages, scroll, and switch tabs. They never fill in forms or change settings.

Screenshots: a picture of every page

```
/webapp-capture screenshots http://localhost:3000
```

What happens: Claude finds every page in your app's main menu and takes pictures from the top to the bottom of each page, including each tab.

You get: `library/design/screenshots/<page>/` with files named after the page, like `dashboard-billing-001.png`.

Good for: documentation, app store listings, design reviews, and "before" pictures before a redesign.

Library: every component, documented

```
/webapp-capture library http://localhost:3000 try 3 pages first and show me
```

A **component** is a reusable piece of the screen: a button, a card, a menu, a table.

What happens:

1. Claude measures every element on every page (sizes, colors, fonts, spacing).
2. It groups the ones that look the same.
3. Several AI helpers describe each group at the same time (this is where parallel agents save a lot of time).
4. Claude merges the descriptions into one folder per component.

You get: `library/design/components/<name>/` with a description, the HTML, measured styles and screenshots, plus `candidate.tokens.json`: your app's colors, font sizes and spacing as **design tokens** (named values a designer or AI can reuse).

Tip: start with "try 3 pages first and show me". Check the sample, then ask for the full run.

Audit: what does not match

```
/webapp-capture audit http://localhost:3000 the code is in ./web
```

What happens: Claude compares everything it measured and looks for things that should be the same but are not: seven slightly different whites, buttons drawn five ways, spacing that does not follow a pattern. If you tell it where the code is, it also finds the lines that cause each problem.

You get: dated reports in `library/requirements/reports/`, sorted by how much each problem matters.

shadcn: move your look to shadcn/ui

```
/webapp-capture shadcn http://localhost:3000
```

shadcn/ui is a popular set of ready-made components for React apps. This job maps each of your components to the closest shadcn/ui one.

You get: `library/design/shadcn/SHADCN-MAP.md` (the map and the exact install command) and `globals.css` (a theme that makes shadcn/ui look like your app).

Claude Design handoff

After the library, audit or shadcn job, Claude also builds one zip file: `library/design/<app>—claude-design-handoff—<date>.zip`. Upload it to Claude Design and type "Read CLAUDE-DESIGN-INSTRUCTIONS.md and follow it" to get a brand guide and design system.

Everything at once

```
/webapp-capture all https://staging.yourapp.com
```

Runs library, audit, shadcn and demo in that order, then the handoff zip. Claude reports after each one, so you can stop early.

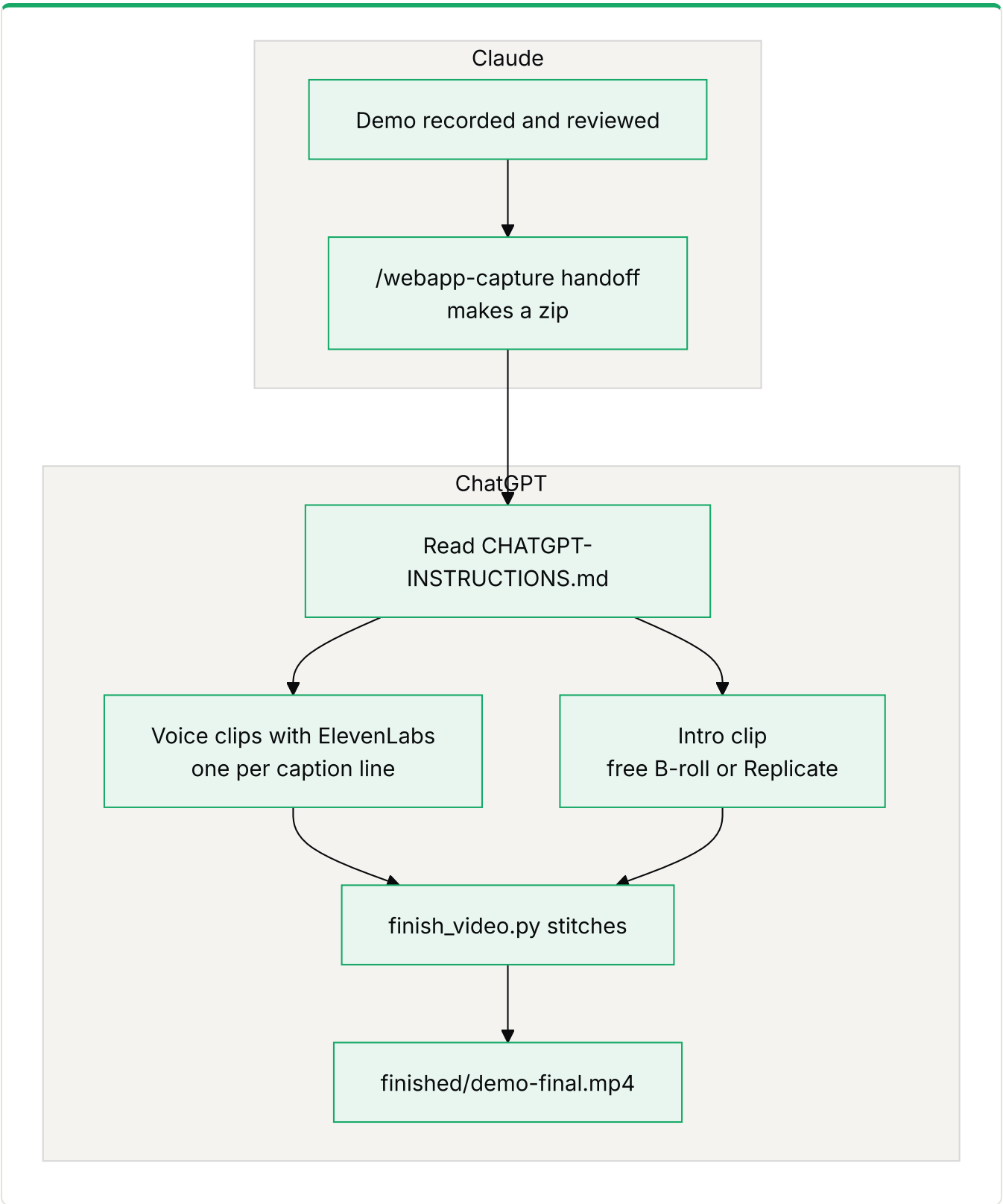
What's next

- Turn this into paid work: 5. Make money with it
- Something went wrong: 6. Fix problems

4. Finish your video in ChatGPT

Claude recorded your app and made a silent video with captions. Now ChatGPT adds three things: a narrator voice, a short intro clip, and the final stitch that joins them. It takes 20 to 40 minutes.

Before you start: you have a finished demo from 2. Your first demo video, including the privacy review.



The voice clips and the intro do not depend on each other, so ChatGPT makes them at the same time. Only the stitch needs both.

What you need

Thing	Notes
The ChatGPT desktop app, using **Work mode** or **Codex**	Local plugins are installed there (OpenAI docs). Whether your plan or workspace can install plugins depends on your plan, role and region (OpenAI help).
An ElevenLabs account	For the voice. **For client work, use a paid plan.** The free plan has no commercial license (ElevenLabs help).
ffmpeg and Python 3.11 or newer	Already done if you followed 1. Install everything
A Replicate account and token	**Only** for AI-made intro clips. Free stock B-roll needs none. See What is Replicate? below.

Step 1. In Claude: make the handoff zip

In Claude Code, type:

```
/webapp-capture handoff
```

Claude asks a few questions about the intro (do you want one, stock or AI-made, what the product does, who it is for, brand colors). Then it builds `<your-demo>-chatgpt-handoff.zip` next to your demo folder.

What is inside: the silent video, the captions, one text file per narrator line, an intro brief, step-by-step instructions for ChatGPT (`CHATGPT-INSTRUCTIONS.md`), and the stitch script (`finish_video.py`).

If it refuses: it will not build the zip until the privacy review exists, because the zip leaves your computer. Run the review first (guide 2, step 9).

Step 2. Install this plugin in ChatGPT (once)

The same plugin folder works in ChatGPT. It gives ChatGPT the intro video skill and the Replicate video tools.

1. Get the plugin's files on your computer. Open the plugin's GitHub page, click the green **Code** button, then **Download ZIP**. Unzip it somewhere you will keep it, like your Documents folder.

   This folder has a plugin list ChatGPT reads (`.agents/plugins/marketplace.json`).

2. Open that folder in the ChatGPT desktop app's Work mode or Codex.

3. Restart the ChatGPT desktop app.
4. Open the **Plugins Directory**, choose **Legion Code Inc. Plugins**, and install **Webapp Capture Video Studio**.

These steps follow OpenAI's local plugin docs (build plugins). OpenAI is still changing this feature, so the menu names may differ slightly in your version.

Step 3. Connect ElevenLabs (once)

1. Go to elevenlabs.io/mcp and use its **Add to ChatGPT** option. You sign in with your ElevenLabs account, so you never paste a key into ChatGPT.
2. If your setup asks for an API key instead, create one at elevenlabs.io/app/settings/api-keys and put it in the connector's settings, never in a chat message.
3. Pick a voice in ElevenLabs that you like. Use a stock voice, or a voice clone the client gave written permission for. Never copy a real person's voice without consent.

Step 4. Replicate token (only for AI-made intros)

Skip this step if you will use free stock B-roll.

What is Replicate?

Replicate is a website that runs AI models for you in the cloud. Video models like Google Veo, Kling and Wan need huge, expensive computers. Replicate owns those computers and rents you a few seconds of time on them. This plugin talks to Replicate for you, so you never have to use its website to make a clip.

Do I need it? Only if you want an AI-made intro clip. Free stock B-roll does not need Replicate. Everything else in this plugin works without it.

What does it cost? You pay only for what you use (billing). Each model shows its price on its page on replicate.com, and video models cost much more than image models, so check the price before you render. A few models can be tried for free, but after a bit Replicate asks you to set up billing (a card, or prepaid credit). If a run fails, you are not charged. If you cancel a run partway, you may still be charged for the time it ran. The plugin always asks before any paid render.

Get your Replicate token (about 3 minutes)

A token is a long password that lets this plugin use your Replicate account. It starts with `r8_`. Treat it like a bank card number.

1. Go to replicate.com and sign up (or sign in).
2. Set up billing on the billing page (a card, or prepaid credit). Video models will not run without it.
3. Open replicate.com/account/api-tokens. Replicate makes a default token when you sign up. You can use it, or click to create a new one named `webapp-capture` so you can turn it off later without breaking anything else.
4. Copy the token.
5. Save it in **one** of these places. Never paste it into a chat with Claude or ChatGPT.
    - **Claude desktop app (or Claude Code):** open the **Terminal** panel (title bar, or Ctrl and backtick), run `claude plugin configure webapp-capture@legioncodeinc`, and paste the token into the private field. Claude keeps it in your computer's secure credential store. Start a new session.
    - **ChatGPT:** open your computer's Terminal app (or the Claude app's Terminal panel), go to this plugin's folder, and run `python3 mcp/replicate-video/scripts/set_token.py --check`. Paste the token when it asks. The typing is hidden. The `--check` part makes one free, read-only call to confirm the token works. It saves the token to `~/.config/webapp-capture/replicate.env`, a file only you can read. Restart the app. Not sure where the plugin folder is? Ask Claude or ChatGPT: "Where is the webapp-capture plugin folder on this computer?"
    - **Advanced:** set `REPLICATE_API_TOKEN` in the environment that starts the app. It wins over the saved file.
6. Check it: ask Claude or ChatGPT to "check the Replicate video connection without generating anything." This runs the production doctor. It says whether a token is there but never shows it.

Lost it or leaked it? Delete that token on replicate.com/account/api-tokens, make a new one, and save it again. To remove it from this computer, run `python3 mcp/replicate-video/scripts/set_token.py --remove`.

Step 5. In ChatGPT: finish the video

1. Unzip the handoff zip.
2. In ChatGPT (Work mode or Codex), open the unzipped folder, and type:

```
Read CHATGPT-INSTRUCTIONS.md and finish this demo video. Use free stock B-roll for
the intro unless I say otherwise. Ask me before anything that costs money.
```

3. Answer ChatGPT's questions: which voice, and which intro clip you like.

What ChatGPT does:

1. Makes one voice clip per caption line with ElevenLabs, and checks each one fits its line.

2. Finds 2 or 3 free B-roll clips for you to pick from (or, if you asked, renders one with a Replicate model after telling you the cost).
3. Checks the intro frame by frame for warped hands, melted objects or fake text.
4. Runs `python3 finish_video.py --intro intro/intro.mp4`. That puts the intro first, lines up each voice clip with its caption, moves the captions to match, and saves `finished/demo-final.mp4`.

Step 6. Check the final video

Open `finished/demo-final.mp4` and watch it all the way through with sound.

- Each voice line should match what is on screen.
- Captions should line up with the voice.
- The intro should be clean, with no logos or strange frames.

Want a change? Tell ChatGPT in normal words, for example "Line 3 is too long. Shorten it, make the voice again, and stitch again." The stitch never overwrites, so ChatGPT renames the old `finished` folder first.

Free B-roll sites

Most intros do not need custom footage. These sites let you use clips in paid client videos for free (checked 2026-10-23):

Site	Free for client work	Credit needed	Good to know
Pexels	Yes (license)	No	Has a free API an AI agent can search. Up to 4K.
Pixabay	Yes (license)	No	Has a free API. No clips with brands for commercial use. Up to 4K.
Mixkit	Yes, for clips marked Free License (license)	No	Some clips are personal use only, so check the label. Do not bulk download with scripts.
Coverr	Yes (license)	No	Logos in clips need separate permission.
Vecteezy	Limited (license)	Yes, on free clips	Free clips are capped at a $1,000 production budget.
Videezy	Yes (license)	Yes, on free clips	Add a credit line.

Rules that keep you safe:

- Save the clip link, the creator, the license and the date for every clip you use.

- Skip clips with logos, brand names, or a person who would look like they endorse the product.
- Give the client the finished video, never the raw stock clips.

Need something custom? The `demo-intro-video` skill can generate an intro with a Replicate model instead. That costs money per clip. The full list with API limits is in free-broll-sources.md.

Prompts for ChatGPT

Type this	What happens
`Check the Replicate video connection without generating anything.`	Runs the doctor. Nothing is paid for.
`Find 3 free B-roll clips for a 5 second intro about field service software. Show me before downloading.`	Searches free sites and shows choices.
`Generate a 6 second intro with Veo 3.1 from INTRO-BRIEF.md. Tell me the cost first.`	Writes the prompt and waits for your yes before the paid render.
`Line 3 is too long. Shorten it, regenerate the voice, and stitch again.`	Fixes one line and rebuilds.

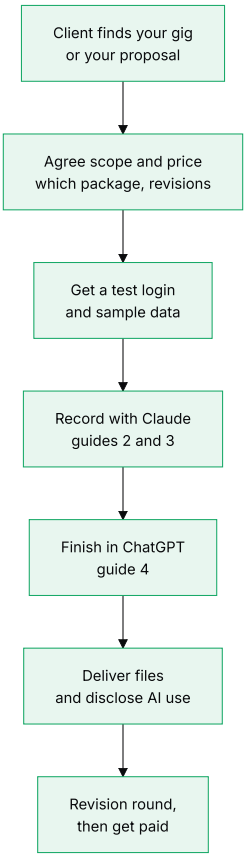
What's next

- Sell this work: 5. Make money with it
- Something went wrong: 6. Fix problems

5. Make money with it

People and companies pay for demo videos, screenshot sets, UI audits and design system work. This plugin does most of the work, so you can sell it as a service on Upwork or Fiverr. This guide shows what to charge, what the sites take, and how to set up your offers.

How a job goes



1. **Agree the scope in writing.** Which package, how many revisions, and the delivery date.
2. **Ask for a test login, not their password.** A staging site or a demo account with sample data. If they only have a live site, agree in writing what will be on screen.
3. **Record and check.** Follow 2. Your first demo video. Always run the privacy review before you send anything.
4. **Finish.** Follow 4. Finish your video in ChatGPT for the voice and intro.
5. **Deliver.** Send the final video, the screenshots or reports, and a short note on which parts used AI.

6. **One revision round,** as agreed, then mark the order complete.

People pay for demo videos, screenshot sets, UI audits and design system work. This plugin does most of the hard part, so you can sell it as a service. The prices below come from real Upwork and Fiverr listings checked on 2026-10-23. Prices change, so check them before you set yours.

What to charge

Service	Fiverr (starting prices seen)	Upwork (seen)	Suggested price
60 to 90 second demo video with captions	$10 to $390, middle about $70 (listings)	$75 to $350 catalog tiers, one real job paid $1,500 (catalog)	$250 to $900 on Fiverr, $400 to $1,500 on Upwork
App walkthrough or tutorial video	$5 to $240, middle about $30 (listings)	$49 to $1,000 (catalog)	$150 to $600
AI voiceover add-on	$5 to $80 (category)	about $40 add-on	$25 to $75 extra
UI/UX audit of a web app	$10 to $500, middle about $73 (listings)	$500 to $1,800, top sellers up to $4,200 (catalog)	$350 to $1,800
Component library or design system docs	$120 to $900 (example gig)	UI designers $20 to $40 an hour (rates)	$500 to $2,500
shadcn/ui migration map	no direct gigs found	no direct listings found	$400 to $2,000 as a premium add-on (estimate)
Screenshot set for docs or app stores	$5 to $625, middle about $35	$40 to $80 catalog tiers	$75 to $300
Everything as one bundle			$900 to $2,500 on Fiverr, $1,500 to $4,000 on Upwork (estimate)

"Suggested price" is our estimate built from the listings. Fiverr tag pages show each seller's cheapest package, so real orders are usually higher.

Fees you pay

- **Fiverr** keeps 20%, so you get 80% of every order and tip. Money clears after 14 days (7 for Top Rated sellers). Buyers pay their own 5.5% fee, plus $3.50 on orders under $200. (Fiverr payment terms)

- **Upwork** takes a fee from 0% to 15% per contract. You see the rate before you accept. It is lower on Direct Contracts with clients you bring yourself. (Upwork fee) Sending a proposal costs Connects, which are $0.15 each. (Connects)

How to sell it

1. **Make three packages on Fiverr.** For example:

   - Basic: screenshot set plus a 30 second clip.
   - Standard: a 60 to 90 second narrated demo with captions.
   - Premium: Standard plus the component library, the UI audit and the shadcn/ui map.

   Start Basic at $200 or more, so buyers skip the small-order fee. (Creating a gig)

2. **Sell extras instead of unlimited changes.** Offer extra revisions, rush delivery, a second language, or an intro clip. (Gig extras)

3. **On Upwork, list the bundle in the Project Catalog** with three tiers, and bid only on jobs that fit, because each bid costs Connects.

4. **Build a portfolio from a sample app.** Show one demo video, one page of an audit, a design token sheet and part of a shadcn/ui map. Use your own sample app or a client who agreed to it.

5. **Set the revision rules up front.** For example: two rounds on the script before recording, and one round on the final cut. A redesign of the app means a new order.

6. **Be honest about AI.** Fiverr requires you to follow a client's "no AI" request and to say how you work when asked. Misleading clients can get your account banned. Use stock AI voices or a voice the client approved. Never copy a real person's voice without consent. (Fiverr AI rules) Upwork bars AI use when your client restricts it. (Upwork terms) Simple rule: write "AI-assisted capture, narration and intro" in every offer.

7. **Use test data.** Record on a staging copy or a demo account, not the client's live customers. The plugin's privacy review helps, but you are still responsible for what is on screen.

8. **Use paid licenses for client work.** ElevenLabs' free plan has no commercial license, so use a paid plan for voices you sell (ElevenLabs help). Free stock B-roll must follow each site's license (free B-roll sources).

9. **Count your costs.** Voice (ElevenLabs) and generated intros (Replicate) cost money per use. Free stock B-roll does not. Add paid costs to the price, or pass them through.

This section is general business information, not legal or tax advice.

6. Fix problems

Find what you see in the left column, then do what the right column says. Most fixes are one sentence you type to Claude.

First rule: when something breaks, type `/webapp-capture doctor` (in Claude) or "check the Replicate video connection without generating anything" (in ChatGPT). Both list the problem and the fix.

Installing

What you see	What to do
No **Code** tab in the app	Update the Claude desktop app and check that your plan includes Claude Code (Pro, Max, Team or Enterprise).
`command not found: claude` in the Terminal panel	Close the panel, open it again, and retry. Still broken? Ask in the Code tab: "why can't the terminal panel find the claude command?"
`/webapp-capture` does nothing or is unknown	The plugin is not installed or not loaded. Redo 1. Install everything, step 3, then start a new session. Plugins do not load in cloud or WSL sessions.
The doctor says **no Chromium**	Type: "install the browser the doctor asked for". Claude runs `npx playwright install chromium`.
The doctor says **ffmpeg not found**	Type: "install ffmpeg for me". Then start a new session and run the doctor again.
The doctor says **origin unreachable**	Your app is not running, or the address is wrong. Open the address in your normal browser to check.

Logging in

What you see	What to do
The login window closed or **timed out**	Type: "run the login step again". Log in within the time shown and do not close the window yourself.
Every page errors with **session expired**	Your login ran out. Type: "run the login step again".
The doctor shows an early cookie expiry	Usually fine. Many apps renew your login quietly. Only log in again if pages start failing.

What you see	What to do
Pictures show the login page instead of the app	Your app keeps you logged in only while the browser is open. Ask Claude to "test one page before the full run".

Demo videos

What you see	What to do
Claude will not record	The plan is not approved yet. Read it and type "yes, approved".
Refusing to rerun: writes would repeat	An earlier take already changed your data. Reset your test data (or let Claude pick new free slots), then type "Retake the demo".
A scene says **mismatch**	The change did not happen the way the plan said. Open `findings.md` and the screenshot in `failures/`. Tell Claude what you see. It is either the app, the test data, or the plan.
A scene says **refusing destructive-looking demo control**	The plan tried to click something like Delete or Log out. That is blocked on purpose. Change the plan to avoid that button.
The demo hangs on a page	The page keeps loading forever (live updates). Ask Claude to "wait for the element instead of the page load".
OVERRUN in the narration fit	A line is too long for its scene. Type: "Shorten the narration in scene 2 and rebuild the captions".
Private info shows in the contact sheets	Tell Claude what to hide (for example "mask the email column"), then retake.
`make-video.sh` **refused**	The final files already exist. Ask Claude to "make the video in a fresh copy of the folder".

ChatGPT, voice and intro

What you see	What to do
The handoff refuses: run review-demo first	Run the privacy review (guide 2, step 9), check the pictures, then run `/webapp-capture handoff` again.
Missing narration/audio/...	A voice clip was not saved. Ask ChatGPT to make the missing line again.
A voice clip **is too long** for its line	Ask ChatGPT to shorten that line and make it again.
The doctor says **no Replicate token**	Follow Get your Replicate token. Restart the app after saving it.

What you see	What to do
Replicate says payment or billing is needed	Set up billing at replicate.com/account/billing.
The intro has warped hands, melted objects or fake text	Do not use it. Ask for a new render with a simpler prompt, or use free stock B-roll instead.
`finished/` already exists	The script never overwrites. Rename the old `finished` folder, then run it again.

Files and folders

What you see	What to do
Files ended up in `library/design/library/design`	The output path in `capture.config.json` repeated the folder name. Use the short path the doctor suggests.
Can't find the video	Look in `library/design/demo/<title>/demo.mp4`, or ask Claude "where is the demo video?"

Still stuck?

- Set `WEBAPP_CAPTURE_DEBUG=1` before a script to see the full error.
- Read SUPPORT.md for how to report a bug. Include what you typed, what you saw, and the doctor output. Never include tokens, passwords or customer data.

Reference

Details for when you want to know exactly what happens. The full reference is USAGE-GUIDE.md in the plugin's docs folder.

What it installs

Nothing is installed globally except the Chromium build Playwright uses. Everything else stays inside the plugin folder or your project.

What	Where	When
The `/webapp-capture` command, the `webapp-capture-stinger` skill, and the `webapp-capture-worker-bee` agent	Your Claude Code plugins folder	When you install the plugin
`playwright-core` and `sharp` (npm)	The skill's `scripts/node_modules/`	First run (`npm install` in `scripts/`)
Chromium for Playwright	Playwright's browser cache (for example `~/Library/Caches/ms-playwright` on macOS)	First run, only if `doctor` finds no Chromium
ffmpeg	Your system package manager	You install it; needed only to assemble demo videos and contact sheets
`capture.config.json`	Your project, next to the outputs (default `library/design/`)	Setup, filled in from your answers
Saved login session (`auth.json` and its `.session.json` sidecar)	A gitignored folder in your project, for example `.capture/`	When you log in once in the browser window
`.gitignore` lines for the session and raw capture data	Your project's `.gitignore`	Setup; Claude shows you the lines it adds
The `replicate_video` MCP server (Python, standard library only)	The plugin's `mcp/replicate-video/` folder; Claude Code and ChatGPT start it with the plugin	Enabled with the plugin. It does nothing paid until you approve a generation, and it needs a token only to generate

What	Where	When
Your Replicate token (optional)	Claude Code's secure credential store, or `~/.config/webapp-capture/replicate.env` (owner-only)	Only if you set one up; see Replicate token

Requirements

Need	Why	Install
Node 20.9 or newer	Runs the capture scripts	nodejs.org
Chromium for Playwright	Headless browser	`npx playwright install chromium` in the skill's `scripts/` folder (Claude does this when `doctor` asks)
Python 3.11 or newer	Replicate video tools and `finish_video.py`	python.org
ffmpeg	Demo assembly and review sheets	`brew install ffmpeg`, `apt install ffmpeg`, or `winget install ffmpeg`
A running copy of your app	The thing being captured	Local, staging, or a seeded demo account is best

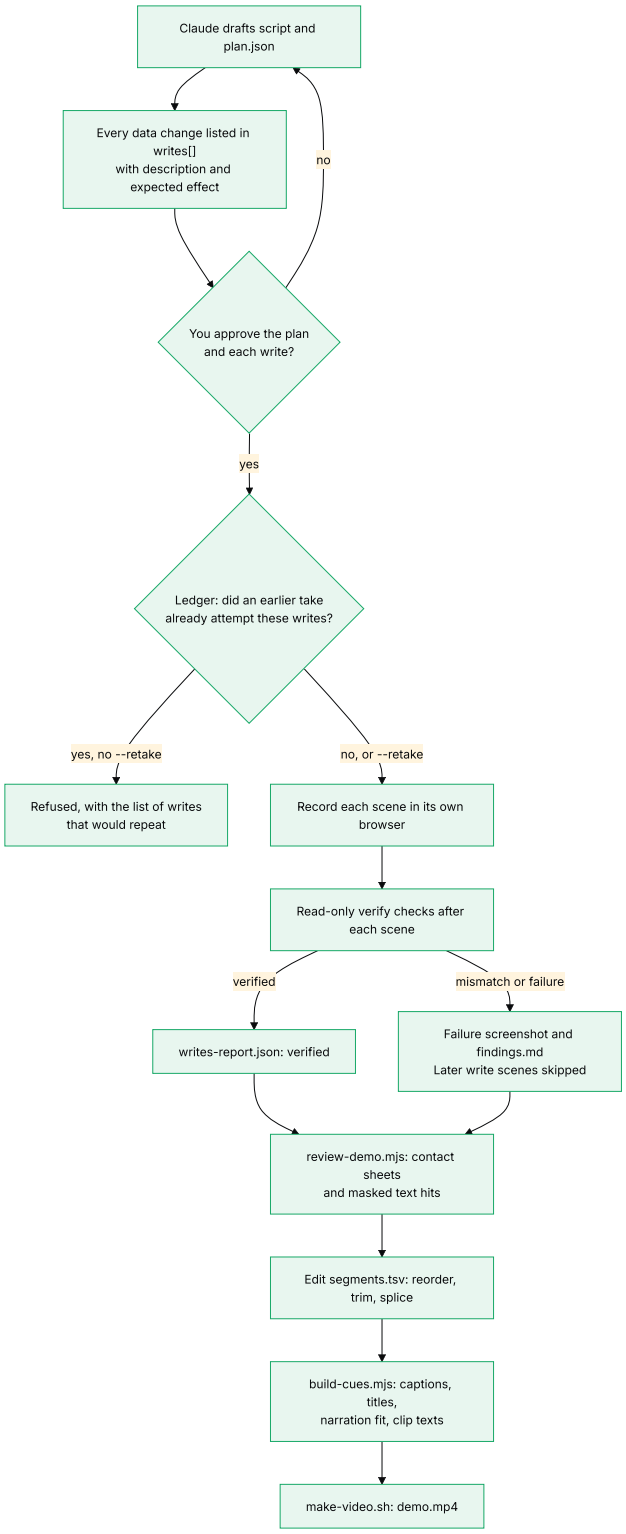
Install the plugin

```
claude plugin marketplace add legioncodeinc/app-auditor-plugin-claude
claude plugin install webapp-capture@legioncodeinc
```

For Claude Cowork, upload the plugin zip from the repository's `dist/` folder. For claude.ai, upload the `.skill` file under Settings, Capabilities, Skills.

Demo route in depth

A demo is the one route that may change your app, so it has the most guard rails. Claude drafts a script and a plan, you approve them, and the recorder proves each change it makes.

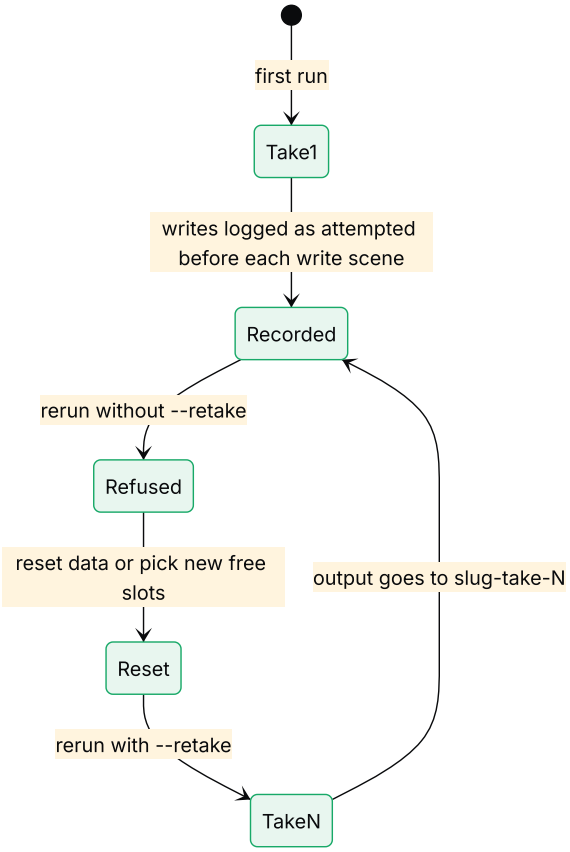

Claude drafts script and plan.json
Every data change listed in writes[] with description and expected effect
no
You approve the plan and each write?
yes
Ledger: did an earlier take already attempt these writes?
yes, no --retake
no, or --retake
Refused, with the list of writes that would repeat
Record each scene in its own browser
Read-only verify checks after each scene
verified
mismatch or failure
writes-report.json: verified
Failure screenshot and findings.md
Later write scenes skipped
review-demo.mjs: contact sheets and masked text hits
Edit segments.tsv: reorder, trim, splice
build-cues.mjs: captions, titles, narration fit, clip texts
make-video.sh: demo.mp4

Plan actions

Action	What it does
`wait`, `hover`, `click`, `type`, `scroll`	The basics. Clicks and typing are checked against the deny list, and credential fields are always refused
`select`	Picks a dropdown option by its visible label
`check`, `uncheck`	Ticks or unticks a checkbox, for example untick Repeats
`drag`	Pointer drag between elements or points, in small steps, for drag-and-drop libraries
`dragTo`	Native HTML drag and drop
`scrollElement`	Scrolls a container such as a calendar grid, or brings a target time into view
`waitFor`	Waits for live state (a selector or text) instead of a fixed delay
`confirmDialog`	Clicks the named button only if the dialog says what the plan expects; otherwise cancels and stops the scene

After a scene, `verify` checks (visible, hidden, text, count) read the page without touching it.

Writes, takes and retakes



- Every take is remembered in `<output.demo>/<slug>.ledger.json`. Takes never overwrite each other.
- A rerun that would repeat earlier writes is refused and lists them. You decide whether to reset the data and run again with `--retake`.
- Pick slots from live state: wait for a free slot by selector, drop onto that same selector, then verify by counting. Never hardcode "Tuesday 9:00".
- After a write mismatch or failure, later write scenes are skipped so one bad drop does not cascade. Read-only scenes keep running.

Privacy review

Pixels can show what text redaction cannot, such as an email in a hover card. During recording, each page's visible text is scanned for emails, phone numbers, and your `redact.patterns`. Only masked hits are stored (for example `d***@***.com`). Then:

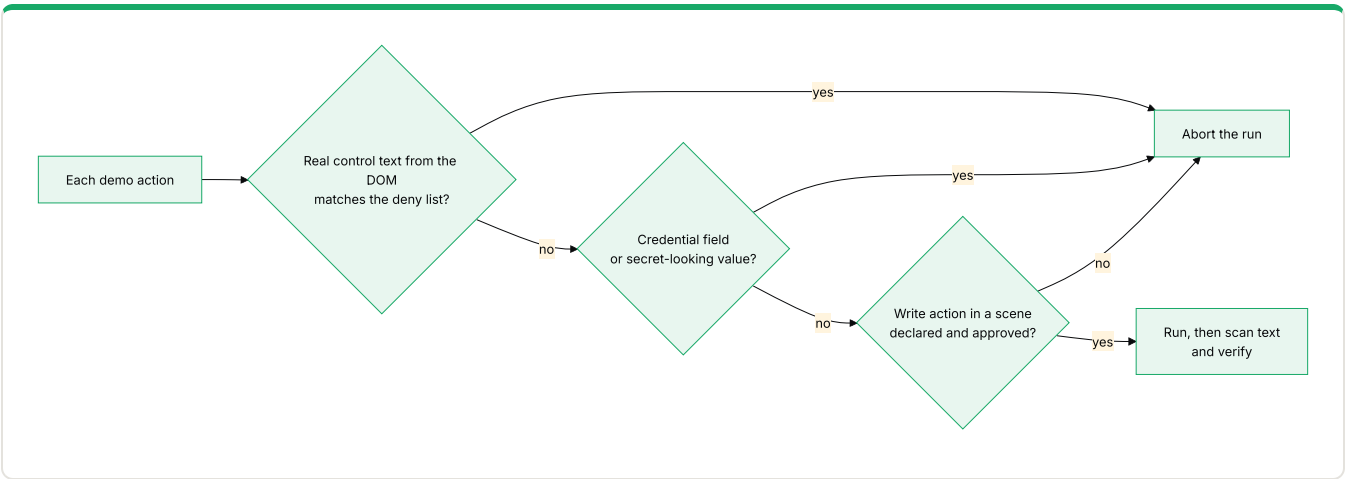
```
CAPTURE_CONFIG=... DEMO_DIR=<demo folder> node <skill>/scripts/demo/review-demo.mjs
```

writes one contact sheet per scene and `review/REVIEW.md`. A person checks every tile before the video is shared.

Editing and narration

- `segments.tsv` is the edit list: one clip per line with in and out points. Reorder, delete, trim, or splice a clip from another take under `clips/`. Failed and mismatched scenes are left out by default.
- `build-cues.mjs` rebuilds `captions.vtt`, `titles.srt`, `narration/fit.md`, and the clip texts after every edit.
- `narration/fit.md` checks each line against the time its scene is on screen at the plan pace (140 words per minute by default) and flags overruns.
- Text to speech for Eleven v4: one clip per line, bracketed tags like `[pause]` and `[warm]` kept, SSML `<break>` converted to tags, and domains spelled out ("app dot example dot com"). No model id is hardcoded; your API key stays in your own secret manager.
- `make-video.sh` refuses linked inputs and existing outputs, cuts and normalizes each segment to 1080p H.264, holds the last frame (`video.endHoldMs`, default 1.5 seconds), and adds soft captions.

How it stays safe



- **You log in, Claude never does.** A real browser window opens and you sign in. The saved session is owner-only and checked for gitignore coverage.
- **Read-only by default.** Capture routes navigate, scroll, and switch tabs. Demos and optional onboarding act only from plans you approve.
- **Built-in deny list always on.** Delete, remove, reset, revoke, logout, pay, purchase, transfer, publish, and similar controls are refused. A plan's own `denyPattern` only adds to the list.
- **Stays on your app.** Navigation to other origins is blocked, popups are closed, and a redirect to the login page stops the run.

- **Secrets stay out.** Text passes redaction patterns, screenshots get solid masks, and every output file is owner-only.
- **Nothing ships silently.** Every run ends with a report and a security, quality, and repository hygiene gate before anything is committed.

Where outputs go

Route	Default location
screenshots	`library/design/screenshots/<route>/`
demo	`library/design/demo/<title>/`, retakes in `<title>-take-N/`, plus `<title>.ledger.json` and `findings.md` beside them
library	`library/design/components/`, `ledger.json`, `candidate.tokens.json`
audit	`library/requirements/reports/`
shadcn	`library/design/shadcn/`
Claude Design handoff	`library/design/<app>-claude-design-handoff-<date>.zip`

Output paths in `capture.config.json` are relative to the config file. If the config lives in `library/design/`, write `demo`, not `library/design/demo`. `doctor` warns when a path repeats the config folder.